

Cas d'étude – Stripe (NoSQL – Data Architecture)

Présentation générale

Entreprise : Stripe

Secteur : Technologies financières (FinTech)

Fondée en : 2010

Siège social : San Francisco, Californie

Produit principal : Plateforme de traitement des paiements pour les entreprises

Contexte

Stripe est un leader mondial des technologies financières.

L'entreprise propose une suite complète de produits permettant de gérer les paiements et les opérations financières des entreprises de toutes tailles.

Elle traite chaque année **des milliards de transactions** et dessert **des millions de marchands** à travers le monde.

Au fur et à mesure de sa croissance, Stripe fait face à une **complexité croissante dans la gestion de ses données**, réparties sur différents types de systèmes :

- systèmes **transactionnels (OLTP)**,
- systèmes **analytiques (OLAP)**,
- et bases de données **non relationnelles (NoSQL)**.

Stripe souhaite construire une **infrastructure de données de nouvelle génération**, intégrant de manière fluide ces trois types de systèmes pour assurer à la fois :

- une **excellence opérationnelle**,
- et une **analyse avancée**.

L'objectif est de créer une **architecture de données unifiée et évolutive**, capable de :

- gérer de grands volumes de données transactionnelles,
 - exécuter des requêtes analytiques complexes,
 - et manipuler des données non structurées (fraude, comportement client, recommandations temps réel).
-

Scénario métier

Avec son expansion internationale, Stripe rencontre plusieurs défis majeurs :

1. Charges transactionnelles complexes (OLTP)

Les systèmes OLTP de Stripe traitent **des millions de transactions par jour**, avec des exigences fortes de disponibilité, cohérence et rapidité.

Les types de transactions incluent : paiements, remboursements, litiges (“chargebacks”) et abonnements.

Stripe doit garantir que ces systèmes soient :

- hautement disponibles,
 - scalables,
 - et capables de supporter de nouvelles fonctionnalités comme la **détection de fraude en temps réel**.
-

2. Besoins analytiques avancés (OLAP)

Stripe doit fournir des **outils analytiques puissants** à ses clients et à ses équipes internes :

- analyse de revenus,
- segmentation client,
- reporting de conformité,
- suivi de performance produit.

Ces analyses nécessitent un système OLAP capable de :

- traiter efficacement des requêtes complexes,
 - supporter des analyses ad hoc,
 - fournir des insights quasi temps réel.
-

3. Gestion de données non structurées et semi-structurées (NoSQL)

Stripe collecte de grandes quantités de données issues de :

- logs,
- retours clients,
- interactions sur le site web et les applications mobiles.

Ces données sont essentielles pour :

- les modèles de **machine learning (fraude, personnalisation, prédiction)**.

Stripe a besoin d'une base NoSQL capable de :

- stocker et traiter efficacement ces données,
 - s'intégrer avec les données relationnelles,
 - et offrir des requêtes flexibles.
-

4. Intégration des systèmes OLTP, OLAP et NoSQL

Un des plus grands défis est d'assurer une **intégration fluide** entre ces trois systèmes :
Les flux de données doivent être :

- **cohérents**,
 - **synchronisés en temps réel**,
 - et **accessibles pour différents usages (analyse, ML, reporting)**.
-

5. Conformité et sécurité

Stripe manipule des **données financières sensibles**, donc la conformité est cruciale.
L'architecture doit respecter :

- **GDPR**,
- **PCI-DSS**,
- **CCPA**.

Des mesures de sécurité robustes sont nécessaires :

- chiffrement (au repos et en transit),
 - gestion des accès,
 - journalisation et surveillance des activités.
-

Exigences métiers

1. Intégrité transactionnelle (OLTP)

Le système OLTP doit :

- gérer des volumes élevés de transactions avec une latence minimale,
- respecter les **propriétés ACID** (atomicité, cohérence, isolation, durabilité),

- prendre en charge les **transactions multi-documents**, la **synchronisation temps réel** et la **reprise après sinistre (disaster recovery)**.
-

2. Analytique performante et scalable (OLAP)

Le système OLAP doit :

- supporter des requêtes complexes sur de grands volumes,
 - agréger les données sur plusieurs dimensions,
 - gérer les **analyses temporelles (time-series)**,
 - effectuer des **jointures et sous-requêtes à grande échelle**,
 - et fournir des résultats à **faible latence** pour des requêtes critiques.
-

3. Gestion flexible des données (NoSQL)

Le système NoSQL doit :

- gérer différents types de données (JSON, XML, binaires),
 - proposer une **conception de schéma flexible**,
 - permettre des **requêtes efficaces sur des structures imbriquées**,
 - et s'intégrer avec les systèmes OLTP et OLAP.
-

4. Intégration et cohérence des données

L'architecture globale doit garantir :

- un **flux cohérent** entre OLTP, OLAP et NoSQL,
 - la gestion de **pipelines batch et temps réel**,
 - des processus **ETL/ELT** performants,
 - des capacités de **transformation et d'enrichissement** de données.
-

5. Sécurité et conformité

L'architecture doit inclure :

- **chiffrement**,

- **contrôle d'accès basé sur les rôles,**
- **audit des accès et des modifications.**

Elle doit également :

- respecter les **normes internationales,**
 - proposer des **rapports de conformité automatisés,**
 - et des **outils de surveillance en temps réel.**
-

Sources de données

1. Données transactionnelles (OLTP)

Toutes les transactions financières traitées par Stripe :

- ID de transaction,
 - ID marchand,
 - ID client,
 - Montant, devise, méthode de paiement,
 - Date et heure,
 - Localisation (IP),
 - Type d'appareil,
 - Statut (réussie, échouée, remboursée),
 - Indicateurs de fraude (scores, anomalies...).
-

2. Données analytiques (OLAP)

Données agrégées et historiques pour les rapports :

- métriques de revenus (jour, semaine, mois),
 - segmentation client,
 - performance produit,
 - analyses de fraude,
 - logs de conformité et d'audit.
-

3. Données semi-structurées et non structurées (NoSQL)

Collectées à partir de différentes sources :

- logs d'erreur et d'accès,
 - données d'interactions (clickstream, sessions),
 - features pour modèles ML,
 - retours clients (avis, enquêtes).
-

4. Données de référence

Données stables ou évoluant lentement :

- codes pays et régions,
 - taux de change,
 - informations marchands,
 - catalogues produits.
-



Exigences techniques

1. Modélisation des données

Tu dois concevoir 3 modèles :

- **OLTP** → schéma normalisé, intégrité transactionnelle, performance.
 - **OLAP** → schéma en étoile ou flocon, tables pré-agrégées, optimisation de requêtes.
 - **NoSQL** → schéma flexible pour données non structurées et requêtes rapides.
-

2. Architecture des pipelines

Créer une architecture d'intégration entre les trois systèmes :

- flux **batch** et **temps réel**,
 - transformations et enrichissements,
 - outils recommandés : **Kafka** (streaming) et **Airflow** (orchestration ETL).
-

3. Scalabilité et performance

- Architecture **horizontale**,
 - Indexation, partitionnement, mise en cache,
 - Bases distribuées et **sharding** pour la haute disponibilité.
-

4. Cohérence et synchronisation

- Garantir la **cohérence des données** entre systèmes,
 - Utiliser **CDC (Change Data Capture)** pour la synchronisation temps réel,
 - Gérer les conflits dans les environnements distribués.
-

5. Sécurité et conformité

- Chiffrement des données,
 - Contrôles d'accès par rôle,
 - Journaux d'audit,
 - Conformité aux normes (GDPR, PCI-DSS),
 - Rapports automatisés de conformité.
-

6. Analytique avancée & Machine Learning

- Intégrer des modèles ML dans le système NoSQL,
 - Supporter la **détection de fraude en temps réel**,
 - Gérer la **personnalisation client**,
 - Fournir des mécanismes de **surveillance et mise à jour** des modèles.
-

Mission

En tant que membre de l'équipe Data Engineering de Stripe, ta mission est de :

Concevoir une **architecture de données complète** intégrant OLTP, OLAP et NoSQL, répondant aux besoins de Stripe en matière de :

- cohérence transactionnelle,

- analytique avancée,
 - flexibilité de gestion des données,
 - et conformité réglementaire.
-

Livrables attendus

1. **Diagramme d'architecture complet**
(OLTP + OLAP + NoSQL + flux de données).
2. **ERD OLTP**
Schéma relationnel normalisé pour le système transactionnel.
3. **Schéma OLAP**
(étoile/flocon, agrégations, vues matérialisées, stratégies de requêtes).
4. **Modèle NoSQL**
Conception flexible (relations, index, embedding vs referencing).
5. **Architecture de pipeline de données**
Description technique (outils, technologies, processus d'intégration).
6. **Plan de sécurité & conformité**
Stratégie de chiffrement, gestion des accès, conformité réglementaire.
7. **Stratégie d'intégration ML**
Processus d'intégration des modèles ML (fraude, personnalisation, monitoring).
8. **Requêtes SQL & NoSQL**
Exemples concrets : revenus, fraude, segmentation client, etc.